UNIT KOMPETENSI 07 • TI.OO.07.2026

MENGIMPLEMENTASIKAN PEMROGRAMAN BERORIENTASI OBJEK (OOP)

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Penerapan Class • Object • Encapsulation • Inheritance • Polymorphism • Interface & Package		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Mengimplementasikan Pemrograman Berorientasi Objek (OOP)** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Penerapan Class**: Mendefinisikan blueprint class ES6 untuk entitas kesalahan sistem kustom (Custom Error Class) dan service helper.
- **Object**: Mengonstruksi instance objek, enkapsulasi Data Transfer Objects (DTO), serta manipulasi object keys/values.
- **Encapsulation**: Menyembunyikan detail implementasi internal, hashing data sensitif, dan pembatasan akses scope lokal modul.
- **Inheritance**: Menerapkan pewarisan properti dan method dari kelas induk native (class ApiError extends Error).
- **Polymorphism**: Menghadirkan perilaku multi-bentuk pada komponen UI adaptif dan penyeragaman format respon RESTful API.
- **Interface & Package**: Menyusun kontrak antarmuka API terstandarisasi serta modularitas packaging (CommonJS vs ES Modules).

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Dewan Penguji yang saya hormati, paradigma **Pemrograman Berorientasi Objek (OOP)** memberikan keunggulan dalam modularitas, pewarisan fungsionalitas, dan keteraturan data. Pada unit kompetensi ini, saya akan mendemonstrasikan implementasi 6 konsep inti OOP pada arsitektur proyek Website Intersis."

[Menjelaskan Sub 1, 2, & 4: Class, Object, dan Inheritance] "Pertama, terkait *Penerapan Class, Object, dan Inheritance*. Kami merancang kelas kustom `ApiError` yang mewarisi (extends) dari kelas dasar JavaScript `Error`. Dengan inheritance ini, `ApiError` mewarisi kapabilitas penangkapan stack trace bawaan, sembari menambahkan atribut baru seperti `statusCode` dan `isOperational`. Setiap kali terjadi kesalahan validasi bisnis, kami menginstansiasi objek baru dari kelas tersebut (`new ApiError(404, 'Data tidak ditemukan')`), sehingga penanganan error di seluruh controller menjadi sangat terstruktur dan seragam."

[Menjelaskan Sub 3: Encapsulation] "Kedua, *Encapsulation (Pembungkusan & Information Hiding)*. Kami menerapkan enkapsulasi pada pengelolaan data kredensial pengguna. Password pengguna tidak pernah disimpan atau diakses dalam bentuk teks biasa (plain text), melainkan dienkapsulasi menggunakan algoritma `bcrypt` dengan salt rounds sebanyak 10 putaran. Selain itu, scope variabel di dalam controller terisolasi dalam modul dan hanya mengekspos fungsi yang dideklarasikan secara eksplisit."

[Menjelaskan Sub 5: Polymorphism] "Ketiga, *Polymorphism (Multi-Bentuk)*. Pada sisi frontend, komponen UI tombol dan kartu dibuat bersifat polimorfis. Komponen `Button` yang sama dapat berubah bentuk, warna, dan perilakunya (sebagai primary action dengan efek gradient, outline border, atau danger button) hanya dengan mengubah properti `variant` yang diteruskan, tanpa perlu menulis tiga komponen tombol terpisah."

[Menjelaskan Sub 6: Interface & Package] "Keempat, *Interface & Package*. Kami menyepakati 'API Contract' terstandarisasi sebagai interface komunikasi antara backend dan frontend: setiap payload response selalu berwujud objek dengan struktur `{ success: Boolean, data: Any, message: String }`. Dari sisi packaging, kami memadukan ES Modules (`import/export`) di frontend dengan CommonJS (`require/module.exports`) di backend, menciptakan ekosistem aplikasi yang rapi dan terisolasi."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

7.1 & 7.4. Penerapan Class ES6 & Pewarisan Objek (Inheritance)

Kaitkan ke Codebase: **`backend/src/utils/ApiError.js`**

Pewarisan kelas diterapkan pada sistem manajemen exception untuk menciptakan hierarki error yang mudah diidentifikasi oleh middleware:

backend/src/utils/ApiError.js

```
// Penerapan Class & Inheritance: Mewarisi Kelas Induk 'Error'
class ApiError extends Error {
  constructor(statusCode, message, isOperational = true, stack = '') {
    super(message); // Memanggil konstruktor kelas induk
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    this.status = `${statusCode}`.startsWith('4') ? 'fail' : 'error';

    if (stack) {
      this.stack = stack;
    } else {
      Error.captureStackTrace(this, this.constructor);
    }
  }
}

module.exports = ApiError;
```

7.2 & 7.3. Objek DTO & Enkapsulasi Keamanan Kredensial Pengguna

Kaitkan ke Codebase: **`backend/src/controllers/admin/authController.js`**

Enkapsulasi data sensitif diterapkan menggunakan teknik hashing kriptografis satu arah, menyembunyikan representasi asli sandi pengguna:

backend/src/controllers/admin/authController.js

```
// Enkapsulasi: Menyembunyikan Detail Password dengan Bcrypt Hashing
exports.login = async (req, res, next) => {
  const { username, password } = req.body;

  const [users] = await db.query('SELECT * FROM users WHERE username = ? AND status = "active"', [username]);
  if (users.length === 0) {
    return res.status(401).json({ success: false, message: 'Kredensial tidak valid.' });
  }

  const user = users[0];
  // Verifikasi hash tanpa pernah mendekripsi password ke bentuk asli
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) {
    return res.status(401).json({ success: false, message: 'Kredensial tidak valid.' });
  }

  // Objek Response Bersih: Menghilangkan properti password dari output DTO
  const userResponseDTO = { id: user.id, name: user.name, username: user.username, role: user.role };
  const token = jwt.sign(userResponseDTO, process.env.JWT_SECRET, { expiresIn: '8h' });

  res.json({ success: true, token, user: userResponseDTO });
};
```

7.5 & 7.6. Polimorfisme Komponen UI & Standarisasi Kontrak Interface

Kaitkan ke Codebase: `frontend/src/components/ui/Button.jsx`

Komponen UI polimorfis menyesuaikan perwujudan visual dan fungsi interaksinya sesuai kontrak interface properti yang diterima:

frontend/src/components/ui/Button.jsx

```
// Polimorfisme Komponen: Menyesuaikan Gaya Berdasarkan Prop Variant
export default function Button({ children, variant = 'primary', className = '', ...props }) {
  const baseStyles = 'px-5 py-2.5 rounded-xl font-medium transition-all duration-200 flex items-center justify-center
gap-2';

  const variantStyles = {
    primary: 'bg-gradient-to-r from-blue-600 to-cyan-500 text-white shadow-md hover:shadow-cyan-500/25',
    secondary: 'bg-slate-800 text-slate-200 hover:bg-slate-700 border border-slate-700',
    danger: 'bg-red-600 text-white hover:bg-red-700 shadow-md hover:shadow-red-500/25',
    outline: 'border border-blue-500/40 text-blue-400 hover:bg-blue-500/10'
  };

  return (
    <button className={`${baseStyles} ${variantStyles[variant]} ${className}`} {...props}>
      {children}
    </button>
  );
}
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Penerapan Class & Inheritance: Pembuatan hierarki custom error (ApiError extends Error).
- Enkapsulasi Data Sensitif: Hashing password dengan Bcrypt dan sanitasi objek respon DTO.
- Information Hiding: Isolasi fungsi controller di dalam modul tertutup (CommonJS/ESM).
- Polimorfisme Komponen: Satu komponen UI adaptif mendukung beragam varian presentasi.
- Standarisasi Interface: Kontrak respons JSON konsisten (success, data, message) untuk semua endpoint.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Mengapa Anda memilih membuat custom class ApiError daripada sekadar melempar objek biasa throw { status: 404, msg: 'Error' }?

Jawaban Pembelaan Presenter: Dengan meng-extends kelas Error bawaan JavaScript, objek ApiError tetap mempertahankan captureStackTrace() yang mencatat nomor baris dan berkas lokasi terjadinya kesalahan. Ini sangat krusial saat debugging di log server, sedangkan objek literal biasa tidak memiliki informasi stack trace.

Q2: Bagaimana Anda menerapkan prinsip Encapsulation pada layer database?

Jawaban Pembelaan Presenter: Kami mengisolasi seluruh interaksi SQL di dalam controller dan repository backend. Komponen frontend tidak dapat melihat struktur query SQL mentah maupun kredensial koneksi database, melainkan hanya berinteraksi melalui endpoint REST API yang telah tervalidasi.